



By: Mohamed Alowais (b00091936), Mohamed Alawadhi (b00094286)
Lab 2 – Lexical Analysis

Objectives:

- Understand lexical analysis
- Develop a program to identify and print lexemes and their tokens

Due date: End of the lab. (Only one team member needs to submit.)

Note: “It is considered an academic integrity violation to represent the output of a generative artificial intelligence tool as your work”

Each team must *work independently* and hand in their own original answers. You are *not* allowed to discuss or *share* any solution, or to *copy* from others, from the Internet, or any other source. Plagiarism and cheating will be severely penalized, starting with a zero grade for the assignment. Recall you are bound by the AUS Academic Integrity Code, which you signed when joining AUS.

Exercise 1: Identify Lexemes

[10 marks]

A lexical analyzer is essentially a pattern matcher. A pattern matcher attempts to find a substring of a given string of characters that matches a given character pattern. An input program appears to a compiler as a single string of characters. The lexical analyzer collects characters into logical groupings called words or lexemes and assigns internal codes called tokens to the groupings.

Your task is to write a C++ program that reads a simple “program” consisting of one or more expressions, as per the language grammar below, and prints the lexemes and tokens found. Note that space is irrelevant and should be skipped (i.e., all of ' ', '\t', '\n', and '\r' characters).

Grammar:

```
<program> → <assignment> ";" or <assignment> ";" <program>
<assignment> → <variable> "=" <expression>
<expression> → <term> or <term> <operator> <expression>
<term> → <variable> or <integer>
<operator> → Arithmetic operator "+" or "-" or "*"
<variable> → one or more letters
<letter> → Alphabets "A" to "Z" and/or "a" to "z"
<integer> → one or more digits
```

<digit> → Any number between "0" to "9" (inclusive)

Use the following codes for the tokens: INT for an Integer, ID for a Variable, ASSIGN_OP for the assignment operator, SEP for ';', then ADD_OP, SUB_OP, MULT_OP for addition '+', subtraction '-', and multiplication operators '*', respectively.

Sample inputs and outputs:

Run 1:

Enter a program: res = 450 - 17 ;

The lexemes and tokens found are:

("res", ID) ("=", ASSIGN_OP) ("450", INT) ("-", SUB_OP) ("17", INT) (";", SEP)

Run 2:

Enter a program: value = sum * 3 - offset ; total = value + total ;

The lexemes and tokens found are:

("value", ID) ("=" , ASSIGN_OP) ("sum", ID) ("*", MULT_OP) ("3", INT) ("-", SUB_OP) ("offset", ID) (";", SEP)
("total", ID) ("=", ASSIGN_OP) ("value", ID) ("+", ADD_OP) ("total", ID) (";", SEP)

Note: To help you, here is some C++ code to divide a string with a delimiter. Use provided Lab1.cpp

```
#include <string>
#include <sstream>
int main() {
    string expression;
    cout << "Enter a program \n ";
    getline(cin, expression);
    stringstream ss(expression); // convert an expression into a string stream
    string str;

    while (getline(ss, str, ' ')) { // use space as a delimiter
        cout << str << endl
    }
    return 0;
}
```

```

Code:
#include<iostream>
#include <string>
#include <sstream>
#include<vector>
#include <cctype>

using namespace std;

bool isIdentifier(string str)//check if str is an identifier
{
    for (char c : str){
        c = std::tolower(c);
        if(!((c >= 97) && (c <= 122)))
            return false;
    }
    return true;
}

bool isInteger(string str)
{
    for(char c : str){
        if(!((c >= 48) && (c <= 57)))
            return false;
    }
    return true;
}

bool isPlus(string str)
{
    return str == "+";
}

bool isMinus(string str)
{
    return str == "-";
}

bool isMultiplication(string str)
{
    return str == "*";
}

```

```

bool isAssignment(string str)
{
    return str == "=";
}
bool isSeperator(string str)
{
    return str == ";";
}

int main()
{
    vector<pair<string, string> > lexeme;
    string expression;
    cout << "Enter expression \n ";
    getline(cin, expression);
    stringstream ss(expression); //convert expression into string stream
    string str;

    while (getline(ss, str, ' ')) { //use space as delim for cutting string
        //find valid leximes and save

        //AI ALGORITHM
        if(isSeperator(str))
            lexeme.push_back({str,"SEP"});
        else if(isAssignment(str))
            lexeme.push_back({str,"ASSIGN_OP"});
        else if(isMultiplication(str))
            lexeme.push_back({str,"MUL_OP"});
        else if(isMinus(str))
            lexeme.push_back({str,"SUBTRACT_OP"});
        else if(isPlus(str))
            lexeme.push_back({str,"ADD_OP"});
        else if(isInteger(str))
            lexeme.push_back({str,"INT"});
        else if(isIdentifier(str))
            lexeme.push_back({str,"ID"});
        else
            lexeme.push_back({str,"INVALID"});

    }

    //print lexeme
    for (const auto& pair : lexeme) {
        cout << "(" << pair.first << ", "
            << pair.second << ")" << std::endl;
    }
}

```

```
    return 0;
}
```

```
(base) → Lab2 g++ Lab1.cpp -o Lab1 && ./Lab1
Enter expression
  res = 450 - 17 ;
("res", ID)
("=", ASSIGN_OP)
("450", INT)
("-", SUBTRACT_OP)
("17", INT)
(";", SEP)
(base) → Lab2 g++ Lab1.cpp -o Lab1 && ./Lab1
Enter expression
  value = sum * 3 - offset ; total = value + total ;
("value", ID)
("=", ASSIGN_OP)
("sum", ID)
("*", MUL_OP)
("3", INT)
("-", SUBTRACT_OP)
("offset", ID)
(";", SEP)
("total", ID)
("=", ASSIGN_OP)
("value", ID)
("+", ADD_OP)
("total", ID)
(";", SEP)
(base) → Lab2 g++ Lab1.cpp -o Lab1 && ./Lab1
Enter expression
  x = -5 ;
("x", ID)
("=", ASSIGN_OP)
("-5", INVALID)
(";", SEP)
```